

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
“Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет
по дисциплине
“Сетевое программное обеспечение”

Студент:
Группа ИА331

Р.К. Рубцов

Преподаватель:
:

А.В. Лошкарев

Новосибирск 2026 г.

Содержание

1	Введение	3
2	Архитектура	3
2.1	Компоненты	3
2.2	Схема взаимодействия	3
2.3	Технологический стек	3
3	Реализация сервисов	3
3.1	Comments Service	3
3.2	Users Service	4
4	API Gateway (Nginx)	4
5	Docker Compose	4
6	Сборка и запуск	4
6.1	Локальный запуск	4
6.2	Проверка	5
7	Заключение	5

1 Введение

Данный отчёт описывает выполнение финального проекта по курсу «Сетевое программное обеспечение». Проект представляет собой микросервисную систему управления комментариями (код проекта: `comments-s19`), состоящую из двух сервисов и API Gateway на Nginx.

2 Архитектура

2.1 Компоненты

Финальный проект включает следующие компоненты:

- **Comments Service** (порт 8138) — основной сервис, реализующий CRUD операции для комментариев на FastAPI.
- **Users Service** (порт 8090) — вспомогательный сервис для управления пользователями на FastAPI.
- **API Gateway** (Nginx, порт 8080) — единая точка входа, маршрутизация запросов к микросервисам.

2.2 Схема взаимодействия

```
Client -> Nginx (:8080)
  /api/comments/* -> Comments Service (:8138)
  /api/users/*    -> Users Service (:8090)
```

2.3 Технологический стек

Компонент	Технология	Обоснование
Язык	Python 3.11	Скорость разработки, асинхронность
Веб-фреймворк	FastAPI	Высокая производительность, Pydantic валидация
API Gateway	Nginx	Лёгкий reverse-proxy
Протокол	REST (JSON)	Универсальность
Контейнеризация	Docker (multistage)	Малый размер образов
Оркестрация	Docker Compose	Локальный запуск

3 Реализация сервисов

3.1 Comments Service

Сервис реализует полный CRUD для комментариев. Каждый комментарий содержит поля `id`, `author`, `text` и `created_at`. Данные хранятся in-memory. Эндпоинты:

- GET `/api/comments` — список всех комментариев
- POST `/api/comments` — создание комментария (201 Created)

- GET /api/comments/{id} — получение по ID
- PUT /api/comments/{id} — обновление комментария
- DELETE /api/comments/{id} — удаление (204 No Content)

При отсутствии ресурса возвращается 404. При невалидных данных — 422.

3.2 Users Service

Сервис управляет пользователями с полями `id`, `name`, `email`. Поддерживает:

- GET /api/users — список пользователей
- POST /api/users — создание пользователя
- GET /api/users/{id} — получение по ID

4 API Gateway (Nginx)

Настроен reverse-проxy на порту 8080. Конфигурация маршрутизации:

```
1 server {
2     listen 8080;
3     location /api/comments {
4         proxy_pass http://comments-svc:8138;
5     }
6     location /api/users {
7         proxy_pass http://users-svc:8090;
8     }
9 }
```

5 Docker Compose

Проект запускается одной командой. Файл `docker-compose.yml` описывает три сервиса с пользовательской сетью:

```
1 services:
2   comments-svc:
3     build: ./services/comments-service
4     ports: ["8138:8138"]
5   users-svc:
6     build: ./services/users-service
7     ports: ["8090:8090"]
8   gateway:
9     image: nginx:alpine
10    ports: ["8080:8080"]
11    volumes: [".:/infra/nginx.conf:/etc/nginx/conf.d/default.conf"]
```

6 Сборка и запуск

6.1 Локальный запуск

```
1 cd weeks/week-17
2 docker compose up --build
```

6.2 Проверка

```
1 # Šözdät' kömmentärii
2 curl -X POST http://localhost:8080/api/comments \
3     -H "Content-Type: application/json" \
4     -d '{"author": "s19", "text": "Hello!"}'
5
6 # Pöläcit' spışök
7 curl http://localhost:8080/api/comments
8
9 # Šözdät' pöi'zoväteia
10 curl -X POST http://localhost:8080/api/users \
11     -H "Content-Type: application/json" \
12     -d '{"name": "s19", "email": "s19@example.com}"'
```

7 Заключение

В ходе выполнения финального проекта были спроектированы и реализованы два микросервиса (Comments Service и Users Service) на FastAPI, объединённые за Nginx API Gateway. Проект контейнеризирован с помощью Docker (multi-stage сборка) и запускается одной командой `docker compose up`. Система поддерживает полный CRUD для комментариев и базовые операции для пользователей.